CHAROTAR UNIVERSITY OF SCIENCE & TECHNOLOGY
FACULTY OF TECHNOLOGY AND ENGINEERING
Smt. Kundanben Dinsha Patel Department of Information Technology

Subject Name: Data Structures & Algorithms **Semester:** III
Subject Code: IT259 **Academic Year**: 2024-25

Practical List
(Jun – Dec 2024)

Note:

1. Students have to register in **Hacker Rank, LeetCode**, **GeeksforGeeks** coding platforms.
2. Students are instructed to solve challenges given in the Practical List.
3. Internal evaluation of practical would be based on score of the coding platforms, performance in regular lab sessions and performance in mid-term practical examination.

Sr. No.	Aim of the Practical	Platform	Hours
1.	1.1 Implement linear search using iterative and recursive using an array. 1.2 Left Rotation on Array https://www.hackerrank.com/challenges/array-left-rotation/problem?isFullScreen=true 1.3 Find All Duplicates in an Array https://leetcode.com/problems/find-all-duplicates-in-an-array/ **Practical Assignment** 1.4 Write a program to find non repeating element in an array (Unique element).	VS code HackerRank Leetcode VS code	2
2.	2.1 Implement binary search iterative and Recursive using an array. 2.2 Implement binary search for a long array of integers to find the required element. However, when the array size is quite large, the equation for finding the mid index may give the value which is out of range of integers. Implement the Binary search with a modified equation for finding mid. “the integer overflow problem” with binary search:	VS code	2

	With a vast list of elements, “right” would be a very large value. Suppose your ‘left’ and ‘right’ are 16 bit unsigned integers. That means, they can only have a maximum value of 2^16 = 65536. <u>Example:</u> Consider: left = 65530 and right = 65531 left + right = 131061 (beyond the integer range). It will give garbage value It is known as an integer overflow. **Practical Assignment** 2.3 Kth Missing Positive Number https://leetcode.com/problems/kth-missing-positive-number/description/ 2.4 Search a 2D Matrix II https://leetcode.com/problems/search-a-2d-matrix-ii/	VS code Leetcode Leetcode	
3.	3.1 Implement Sorting Algorithm(s). (a) Bubble Sort (b) Selection Sort (c) Insertion Sort	VS code	2
	Practical Assignment 3.2 Sort-colors (Leetcode) https://leetcode.com/problems/sort-colors/ 3.3 Sort https://leetcode.com/problems/sort-an-array/	Leetcode	
4.	4.1 Implement following operations of singly linked list. (a) Insert a node at front (b) Insert a node at end (c) Insert a node at specific position (d) Delete a node (e) Print nodes in Reverse (f) Traversal of the Linked List	VS code	2

	4.2 Remove Nth Node from End of List https://leetcode.com/problems/remove-nth-node-from-end-of-list/ 4.3 Swap Nodes in Pairs https://leetcode.com/problems/swap-nodes-in-pairs/ **Practical Assignment** 4.4 Middle of the Linked List https://leetcode.com/problems/middle-of-the-linked-list/ 4.5 Reverse Linked List https://leetcode.com/problems/reverse-linked-list/	LeetCode	2
5.	5.1 Implement following operations of doubly linked list. (a) Insert a node at front (b) Insert a node at end (c) Insert a node after given node information (d) Delete a node at front (e) Count number of nodes (f) Traversal of the Linked List **Practical Assignment** 5.2 Merge Two Sorted Lists https://leetcode.com/problems/merge-two-sorted-lists/ 5.3 Merge k Sorted Lists https://leetcode.com/problems/merge-k-sorted-lists/	VS code	2
6.	6.1 Implement stack using array (static stack). https://practice.geeksforgeeks.org/problems/implement-stack-using-array/1 6.2 Implement stack using linked list (dynamic stack). https://practice.geeksforgeeks.org/problems/implement-stack-using-linked-list/1	GeeksforGeeks	2

7.	7.1 Convert Infix to Postfix https://www.hackerrank.com/contests/2022-23-dsa-lab/challenges/convert-infix-expression-to-postfix-expression- 7.2 Evaluate Reverse Polish Notation https://leetcode.com/problems/evaluate-reverse-polish-notation/ 7.3 Valid Parentheses https://leetcode.com/problems/valid-parentheses/	HackerRank LeetCode	2
8.	8.1 Implement queue using array (static queue). https://practice.geeksforgeeks.org/problems/implement-queue-using-array/1 8.2 Implement queue using linked list (dynamic queue). https://practice.geeksforgeeks.org/problems/implement-queue-using-linked-list/1	GeeksforGeeks	2
	Practical Assignment 8.3 Implement Stack using Queues https://leetcode.com/problems/implement-stack-using-queues/ 8.4 Implement Queue using Stacks https://leetcode.com/problems/implement-queue-using-stacks/	LeetCode	
9.	9.1 Implement Binary Tree with following operations. (a) Binary Tree Inorder Traversal https://leetcode.com/problems/binary-tree-inorder-traversal/ (b) Binary Tree Preorder Traversal https://leetcode.com/problems/binary-tree-preorder-traversal/ (c) Binary Tree Postorder Traversal https://leetcode.com/problems/binary-tree-postorder-traversal/ (d) Binary Tree Levelorder Traversal https://leetcode.com/problems/binary-tree-level-order-traversal/	LeetCode	2

10.	10.1 Implement Binary Search Tree (BST) Insertion https://www.hackerrank.com/challenges/binary-search-tree-insertion/problem	Hacker Rank	2
	10.2 Implement searching in Binary Search Tree (BST) https://leetcode.com/problems/search-in-a-binary-search-tree/ **Practical Assignment** 10.3 Kth Smallest Element in a BST https://leetcode.com/problems/kth-smallest-element-in-a-bst/	LeetCode	2
11.	11.1 Implement DFS of Graph. https://practice.geeksforgeeks.org/problems/depth-first-traversal-for-a-graph/1	GeeksforGeeks	2
	11.2 Implement BFS of Graph. https://practice.geeksforgeeks.org/problems/bfs-traversal-of-graph/1 **Practical Assignment** 11.3 Detect cycle in an undirected graph https://www.geeksforgeeks.org/problems/detect-cycle-in-an-undirected-graph/1	GeeksforGeeks	2
12.	12.1 Implementing a Hash Table for Student Records Management Background: You are tasked with implementing a hybrid hash table to manage student records efficiently. Each student record consists of a unique student ID (key) and the corresponding student score (data). The hash table will support two methods of collision handling: separate chaining and linear probing. This flexibility ensures efficient handling of collisions, enabling you to choose the most suitable method based on different scenarios.	VS code	2